RNN / LSTM for Cyber Attack Sequence Detection

Bidirectional LSTM with Attention vs. Simple RNN vs. GRU on System-Call Sequences

---

Task: Design and evaluate an RNN/LSTM model to detect cyber attack sequences from system-call data (4 marks).

Summary of results (test set, mean of 3 seeds)

Model	Parameters	Accuracy	F1-score	Mean latency	False-alarm rate
Simple RNN	2,609	0.9772	0.9464	22.3 syscalls	20.9%
GRU	4,577	0.9828	0.9593	21.2 syscalls	12.1%
BiLSTM + Attention	10,721	0.9881	0.9715	23.8 syscalls	4.2%

Important note on the data. The real ADFA-LD and CICIDS-2017 files could not be downloaded in the environment where this work was run (no internet access). The experiments therefore use a **synthetic, ADFA-LD-style system-call dataset** that I generated (Linux syscall vocabulary, benign process profiles, six attack families with the same names as in ADFA-LD). All numbers in this report are for that synthetic benchmark and must not be quoted as results on the real ADFA-LD. The pipeline (tokenisation, windowing, models, training, metrics) can be reused on real ADFA-LD traces, but real traces only carry a trace-level label, so window labels and the onset-based latency measure would have to be redefined. Section 2 explains exactly how the data was built, and Section 8 lists the resulting limitations.

1. Introduction and Objectives

Host-based intrusion detection systems (HIDS) watch the sequence of system calls issued by running processes. Individual calls such as *read* or *open* are harmless on their own; it is their *order* and *context* that distinguishes, for example, a web server serving files from a web shell spawning a command interpreter. This makes the problem a natural fit for sequence models such as recurrent neural networks (RNNs).

This report covers all the tasks in the brief:

• select a time-series cyber-security dataset (Section 2);
• pre-process sequences by tokenisation, padding and sliding-window construction (Section 3);
• build a Bidirectional LSTM with an attention mechanism for sequence classification (Section 4);
• train with early stopping and plot training / validation loss and accuracy per epoch (Section 5);
• evaluate with Accuracy, F1-score and Mean Detection Latency (Sections 6-7);
• compare the BiLSTM against a simple RNN and a GRU on the same data (Section 7).

Implementation. No deep-learning framework (TensorFlow / PyTorch) was available in the execution environment, so all three networks were implemented from scratch in NumPy, including the forward pass, back-propagation through time, the attention layer and the Adam optimiser. The analytic gradients of all four architectures were verified against numerical finite-difference gradients (maximum relative error below 3e-5), so the training is mathematically the same as in a standard framework. Because the networks are trained on a single CPU core, they are deliberately small (2.6k to 10.7k parameters).

2. Dataset

2.1 Why a synthetic ADFA-LD-style dataset

The ADFA Linux Dataset (ADFA-LD) contains traces of Linux system-call numbers for normal activity and for attacks produced with six tools: Hydra-FTP, Hydra-SSH, Adduser, Java-Meterpreter, Meterpreter and a Webshell. Since the files were not obtainable offline, I built a generator that reproduces the structure of this problem. It is a stand-in for ADFA-LD, **not** a copy of it.

2.2 Generation process

• **Vocabulary:** 129 real Linux system-call names (98 of which occur in the training traces) (read, write, open, mmap, execve, socket, ptrace, ...).
• **Normal traces:** each trace is generated by one of four first-order Markov "application profiles" (network client, file manager, shell/process spawner, timer/event loop). Each profile uses 20 shared core calls plus about 14 profile-specific calls, with sparse, structured transition probabilities. Length is log-normal (median about 490 calls, 4% very short traces of 20-39 calls to exercise padding).
• **Attack traces:** each starts with benign behaviour (60-400 calls, the "pre-attack" phase), then an attack segment of 200-500 calls, and sometimes ends with more benign calls. Inside the attack segment each step is drawn from a mixture: 45% attack-family Markov chain and 55% the benign profile, and with probability 3.5% a short family-specific *motif* (6-9 calls, e.g. *fork, setsid, dup2, dup2, dup2, execve* for Meterpreter) is inserted. Because the attack mixes in benign calls and shares many calls with normal traffic, the classes overlap substantially.
• **Ground truth:** the attack onset and end positions are stored, which allows true per-window labelling and a meaningful latency measurement (real ADFA-LD only gives a label per trace).

2.3 Splits

Split	Normal traces	Attack traces	Windows	Window stride	Attack-window share
Train	500	250	21,318	20	20.0%
Validation	150	75	6,416	20	~20%
Test	300	150	50,303	5	20.7%

Splitting is done **by trace**, never by window, so no window of a test trace is ever seen in training (no leakage). Each split is generated with a different random seed. The test set contains 37 Meterpreter, 27 Webshell, 23 Hydra-SSH, 22 Java-Meterpreter, 22 Adduser and 19 Hydra-FTP traces.

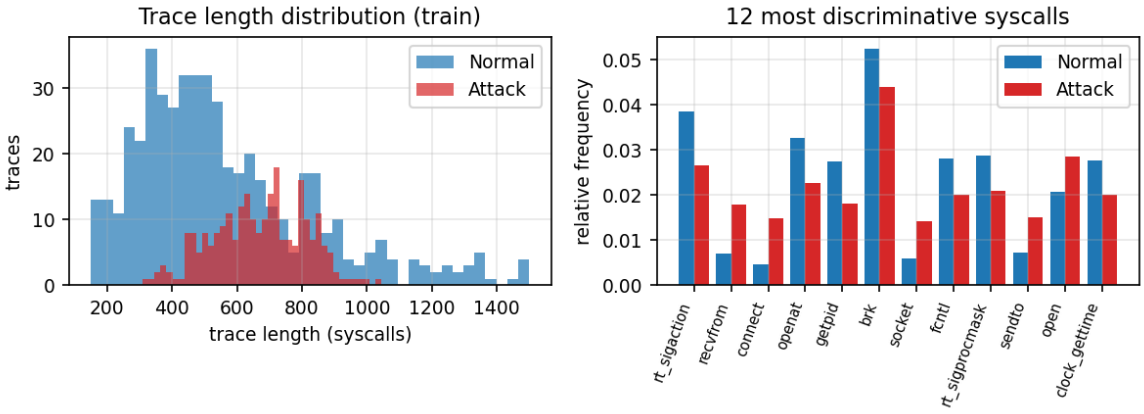


Figure 1. Left: trace-length distributions of the training set. Right: the 12 system calls whose relative frequency differs most between normal and attack traces. The distributions overlap heavily, so no single call decides the class.

3. Pre-processing

Tokenisation. A tokenizer is fitted on the *training* traces only. It assigns each distinct system-call name an integer id and reserves id 0 for <PAD> and id 1 for <UNK> (calls never seen in training). The resulting vocabulary has 100 entries (98 calls observed in training plus the two special tokens). Validation and test traces are encoded with this same vocabulary.

Sliding window. Traces have very different lengths, so each trace is cut into fixed windows of $W = 40$ consecutive calls. Training and validation windows use a stride of 20 (50% overlap); test windows use a stride of 5 so that detection latency can be measured finely. A window is labelled **attack** if at least half of its calls lie inside the true attack segment, otherwise **normal** (all windows of normal traces are normal).

Padding. A trace shorter than 40 calls is front-padded with <PAD> (id 0) so that it still forms exactly one window, with the real calls adjacent to the end of the sequence. The padding embedding is fixed to a zero vector.

Class imbalance. About 20% of training windows are attack windows. The loss is weighted so that both classes contribute equally (positive weight about 4.0).

4. Model Architectures

All models share the same input layer: an embedding of dimension 16, so a window of 40 token ids becomes a 40 x 16 matrix. They differ in the recurrent part, and all end in a single sigmoid unit giving P(attack). Hidden sizes are 24 per direction.

4.1 Bidirectional LSTM with attention (proposed model)

Two LSTM layers read the window in opposite directions. For each time step t the forward and backward hidden states are concatenated, $h_t = [h_t^{fwd} ; h_t^{bwd}]$ (48 values). The LSTM cell is the standard one with input, forget and output gates (forget-gate bias initialised to 1):

$$i,f,o = sigmoid(W\,x_t + U\,h_{t-1} + b),\quad g = tanh(...),\quad c_t = f * c_{t-1} + i * g,\quad h_t = o * tanh(c_t)$$

An additive (Bahdanau-style) attention layer then scores every time step and forms a weighted summary of the window:

$$u_t = tanh(W_a\,h_t + b_a),\quad s_t = v\,.\,u_t,\quad alpha = softmax(s),\quad context = sum_t\,alpha_t\,h_t$$

The context vector goes to the output unit: P(attack) = sigmoid(w . context + b). The attention lets the model concentrate on the few calls in the window (for example an *execve* after a *fork*) that matter most, instead of relying on one final state. Attention size is 24; the model has **10,721** trainable parameters.

4.2 Baselines

Simple RNN (2,609 parameters): $h_t = \tanh(W\,x_t + U\,h_{t-1} + b)$, one direction, classification from the last hidden state.
GRU (4,577 parameters): reset and update gates, $h_t = (1 - z) * n + z * h_{t-1}$, one direction, classification from the last hidden state.

The baselines are the standard unidirectional, last-state versions requested in the brief. Note that this means the BiLSTM differs from them in three ways at once (gating, directionality and attention), and it has more parameters; Section 7 discusses what can and cannot be concluded from that.

5. Training Procedure

Setting	Value
Optimiser	Adam (lr 0.003, beta 0.9 / 0.999), gradient-norm clipping at 5.0
Loss	Class-weighted binary cross-entropy
Batch size	128 windows
Maximum epochs	30
Early stopping	Monitor validation loss; patience 4 epochs; weights of the best epoch are restored
Seeds	3 independent runs (0, 1, 2) per model: different initialisation and batch order
Decision threshold	0.5 on P(attack)

Early stopping triggered in every run. The best epochs were [18, 9, 16] for the BiLSTM+Attention, [9, 9, 13] for the GRU and [11, 11, 9] for the RNN (seeds 0, 1, 2). Figure 2 shows loss and accuracy per epoch for seed 0 of each model; the dashed vertical line marks the epoch whose weights were restored.

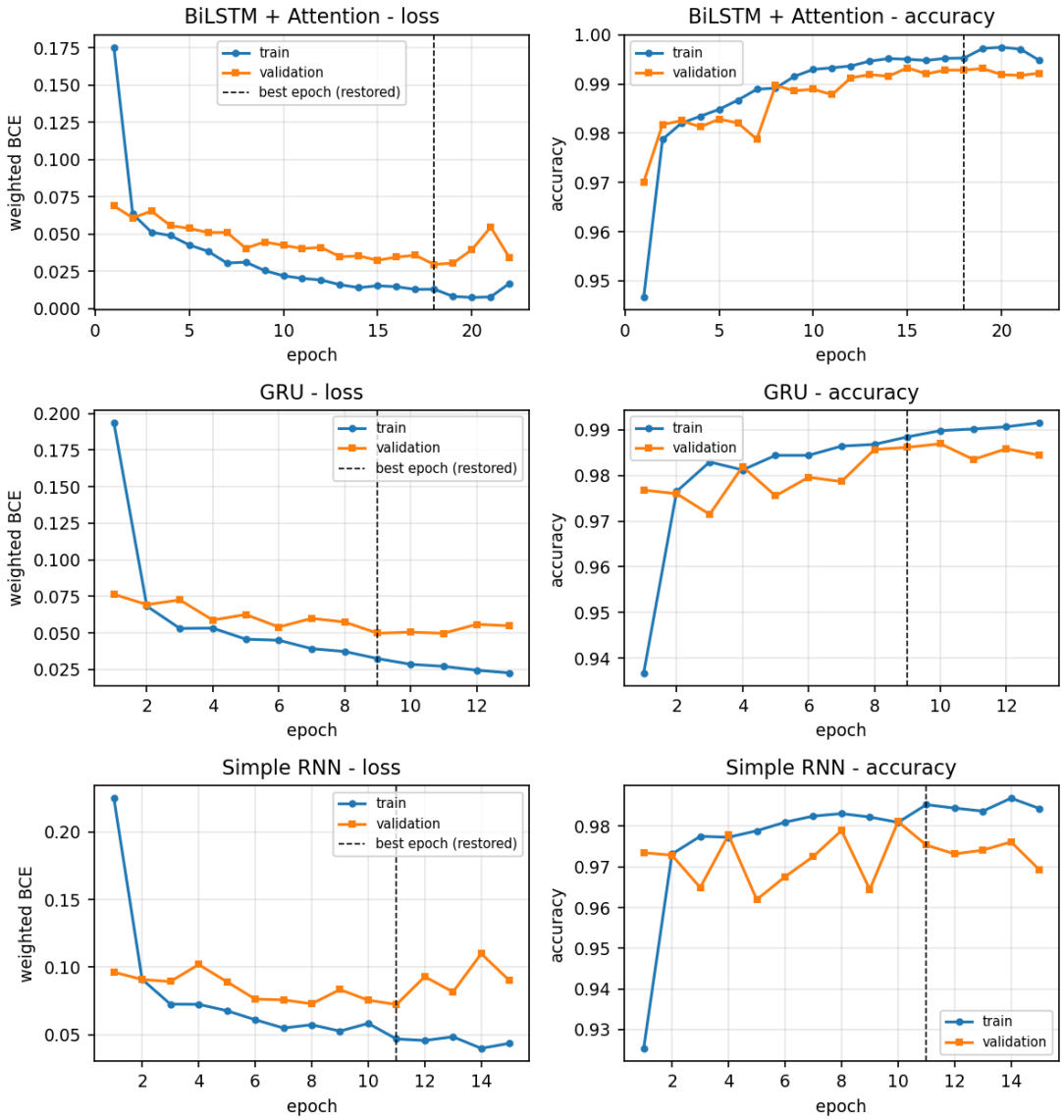


Figure 2. Training and validation loss (left) and accuracy (right) per epoch, seed 0. Training continues to improve after the best epoch while validation loss stalls or rises, which is the over-fitting that early stopping guards against.

The BiLSTM+Attention reaches the lowest validation loss (about 0.03 versus about 0.05 for the GRU and 0.07 for the RNN) and the highest validation accuracy. The simple RNN has the noisiest validation curve, consistent with its weaker ability to retain information over a 40-step window.

6. Evaluation Metrics

Accuracy and F1-score are computed on all 50,303 test windows (stride 5), taking "attack" as the positive class: F1 = 2PR/(P+R). Because attack windows are only about 19% of the test windows, accuracy alone is optimistic, so F1, precision and recall are also reported.

Mean Detection Latency (average number of samples before correct classification) is measured on the 150 attack traces of the test set. A real monitor would not raise an alarm on a single noisy window, so an alert is defined as **2 consecutive windows** classified as attack. For each attack trace the latency is the number of system calls observed *after the true attack onset* up to the end of the window that completes the first alert: latency = (end position of the alerting window) - (onset). Latency is averaged over the traces in which the attack is detected. I also report the detection rate and the **trace-level false-alarm rate** (share of normal traces that trigger the same 2-in-a-row alert), because a detector can trivially have low latency by alerting too eagerly.

Unit note: one "sample" is one system call. With a window of 40 and half of it required to be inside the attack segment for a window to count as an attack window, a latency below about 20 calls is not physically possible for a detector that needs a mostly-attack window, so latencies in the low twenties are close to the practical floor of this set-up.

7. Results and Model Comparison

7.1 Main results

Table 1. Window-level test performance, mean +/- standard deviation over 3 seeds.

Model	Accuracy	F1-score	Precision	Recall
Simple RNN	0.977 +/- 0.002	0.946 +/- 0.005	0.921 +/- 0.011	0.973 +/- 0.003
GRU	0.983 +/- 0.002	0.959 +/- 0.003	0.938 +/- 0.010	0.982 +/- 0.005
BiLSTM + Attention	0.988 +/- 0.003	0.971 +/- 0.007	0.964 +/- 0.013	0.980 +/- 0.000

Table 2. Detection-latency results (latency in system calls after attack onset), mean over 3 seeds.

Model	Mean latency	Median latency	Detection rate	False-alarm rate	Train time (s)	Best epoch
Simple RNN	22.3 +/- 0.4	20.5	100%	20.9% +/- 3.3	8	10.3
GRU	21.2 +/- 0.6	20.5	100%	12.1% +/- 2.8	24	10.3
BiLSTM + Attention	23.8 +/- 0.7	22.7	100%	4.2% +/- 3.9	81	14.3

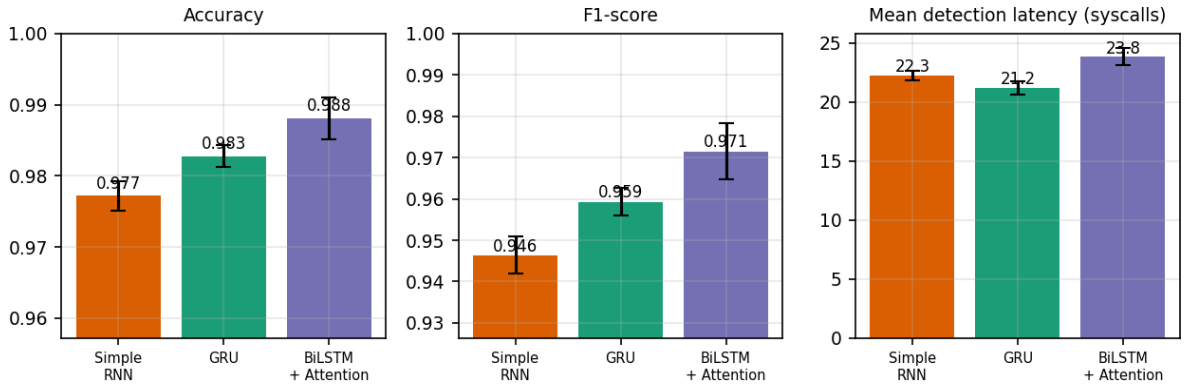


Figure 3. Test accuracy, F1-score and mean detection latency (error bars: 1 standard deviation over 3 seeds).

Table 3. Individual runs.

Model / seed	Accuracy	F1	Mean latency	False-alarm rate
Simple RNN / 0	0.9783	0.9486	22.6	17.0%
Simple RNN / 1	0.9743	0.9401	22.4	25.0%
Simple RNN / 2	0.9790	0.9504	21.7	20.7%
GRU / 0	0.9847	0.9634	21.9	8.3%
GRU / 1	0.9828	0.9595	21.3	13.0%
GRU / 2	0.9809	0.9551	20.4	15.0%
BiLSTM + Attention / 0	0.9903	0.9765	24.4	1.0%
BiLSTM + Attention / 1	0.9839	0.9618	22.8	9.7%
BiLSTM + Attention / 2	0.9901	0.9760	24.4	2.0%

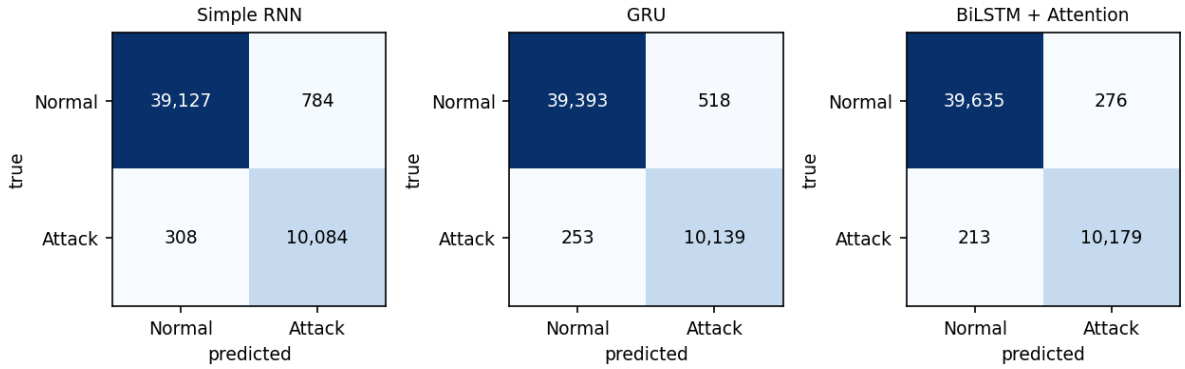


Figure 4. Window-level confusion matrices on the test set (seed 0).

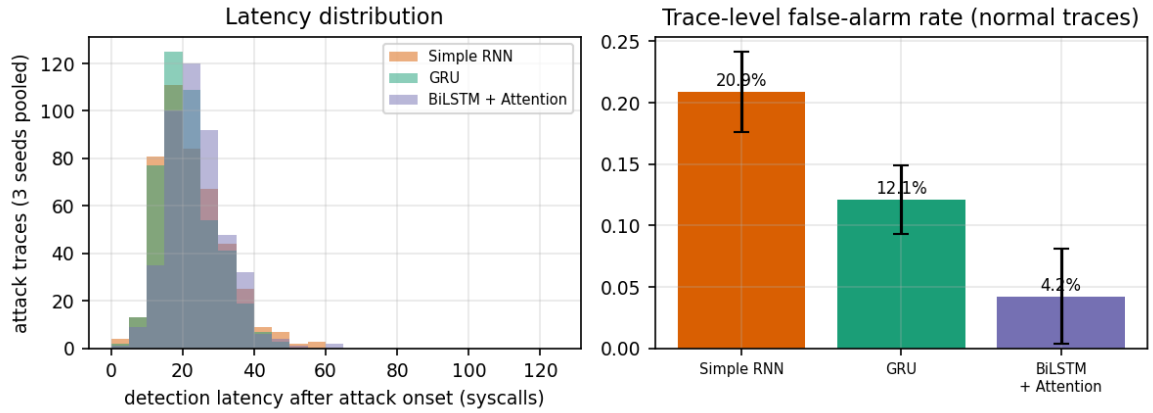


Figure 5. Left: distribution of detection latency across attack traces (3 seeds pooled). Right: share of normal traces that wrongly trigger an alert.

7.2 Discussion

Accuracy and F1. The ranking is the same on both metrics: BiLSTM+Attention (accuracy 0.988, F1 0.971) > GRU (0.983, 0.959) > simple RNN (0.977, 0.946). The BiLSTM's gain over the GRU (about 0.5 points of accuracy and 1.2 points of F1) is larger than the seed-to-seed standard deviation for F1 (0.007) but of similar size for accuracy (0.003), so with only 3 seeds I treat the BiLSTM-over-GRU advantage as suggestive rather than conclusive. The GRU-over-RNN gap (about 1.3 points of F1) is clearer, as the two RNN families do not overlap across seeds.

Precision versus recall. All three models detect attack windows at a similar rate (recall 0.97-0.98). The difference is in precision: the BiLSTM produces roughly half as many false positives as the simple RNN. This is the main source of its F1 advantage.

Detection latency. All models detect 100% of attack traces. Mean latency is similar, 22.3 (RNN), 21.2 (GRU) and 23.8 (BiLSTM) syscalls after onset, i.e. the attack is flagged within about two dozen calls of starting. The BiLSTM is actually about 2.5 calls *slower* than the GRU on average. The false-alarm rates show why this is not a plain loss: the RNN and GRU raise an alarm on 21% and 12% of normal traces, against 4.2% for the BiLSTM. The faster models are partly faster because they are more trigger-happy. At equal false-alarm rates the latency gap would likely shrink or reverse, but I did not run that threshold-matched comparison.

Cost. The BiLSTM has 4x the parameters of the RNN and took about 81 s to train versus 8 s (RNN) and 24 s (GRU) in this NumPy implementation. For a deployed monitor this is still tiny.

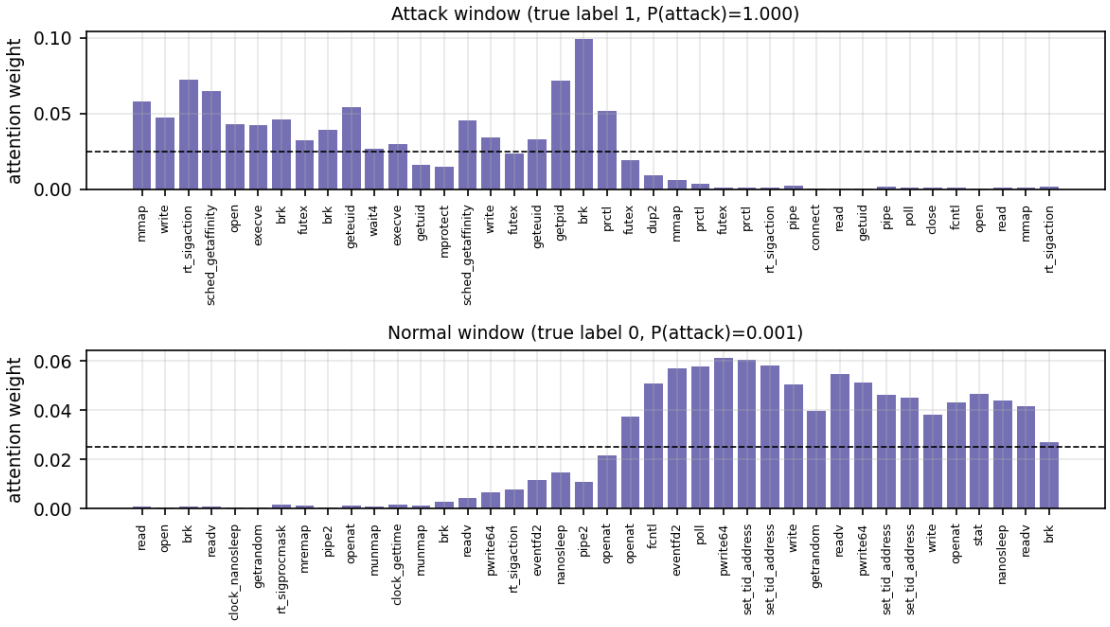


Figure 6. Attention weights of the BiLSTM over the 40 calls of one attack window (top) and one normal window (bottom). The dashed line is the uniform weight 1/40.

What the attention does. Figure 6 shows that attention is concentrated on a handful of positions, well above the uniform level of 0.025. However, averaged over 400 correctly classified windows of each class the peak attention weight is nearly identical for attack windows (0.119) and normal windows (0.112). So in this small model attention acts as a learned pooling mechanism, but I found no evidence that its weights specifically single out attack motifs; I would not present the heat-map as an explanation of the detections without further analysis. I also did not run a BiLSTM *without* attention, so the benefit of the attention layer itself is not separated from the benefit of bidirectionality and gating.

8. Limitations

• **Synthetic data.** The traces come from Markov chains with injected motifs, designed by me. Real ADFA-LD attacks are messier, traces are labelled only at trace level, and the benign class is more diverse. Absolute scores here (about 98-99% accuracy) are very likely higher than on real data, and the ranking of models might change. Results are indicative of the method, not of real-world detection performance.
• **Known onset.** The latency measurement depends on knowing where the attack starts, which is possible only because the data is synthetic.
• **Few seeds, one split.** Three seeds per model and one data split give only rough error bars; no significance tests were run.
• **Confounded comparison.** The BiLSTM+Attention differs from the baselines in several ways at once and has more parameters; an ablation (BiLSTM without attention, a bidirectional GRU, parameter-matched baselines) would be needed to attribute the gain.
• **Small models, fixed hyper-parameters.** Window size, hidden size and learning rate were chosen once and not tuned for any model.
• **Threshold.** A single threshold of 0.5 was used; ROC / precision-recall analysis and threshold matching would give a fairer latency comparison.

9. Conclusion

A Bidirectional LSTM with additive attention was built, trained with early stopping and evaluated against a simple RNN and a GRU for detecting attack sequences in system-call traces, using a synthetic ADFA-LD-style benchmark. On this benchmark the BiLSTM+Attention achieved the best accuracy (0.988)

and F1-score (0.971) and by far the fewest false alarms (4.2% of normal traces), while detecting every attack trace about 24 system calls after the attack began. The GRU was a strong second and the simple RNN the weakest. The BiLSTM's latency was marginally higher than the baselines', which is largely explained by their higher false-alarm rates. The conclusions are limited by the synthetic data and the small number of seeds; the natural next step is to rerun the same pipeline on the real ADFA-LD traces with attention ablations.

Appendix A. Core Implementation (excerpts)

The listings show the BiLSTM-with-attention forward pass and the window / label construction. Back-propagation is implemented by hand for every layer and verified by a numerical gradient check.

```
# ---- tokenisation, padding and sliding windows (data.py) ----
class Tokenizer:
    def fit(s, traces):                       # fitted on TRAIN traces only
        toks = sorted({t for tr in traces for t in tr})
        s.v = {'<PAD>': 0, '<UNK>': 1}
        for t in toks: s.v[t] = len(s.v)
    def enc(s, tr): return np.array([s.v.get(t, 1) for t in tr])

def windows(seq, W, stride, onset=-1, end=-1):
    n = len(seq)
    if n < W:                                 # short trace -> front padding
        x = np.zeros((1, W), int); x[0, W-n:] = seq; ends = np.array([n])
    else:
        st = np.arange(0, n-W+1, stride)
        x = np.stack([seq[i:i+W] for i in st]); ends = st + W
    if onset >= 0:                            # label = >=50% of window inside attack
        lo = np.maximum(ends-W, onset); hi = np.minimum(ends, end)
        y = (np.clip(hi-lo, 0, None) >= W/2).astype(np.float32)
    else: y = np.zeros(len(x), np.float32)
    return x, y, ends

# ---- BiLSTM + additive attention forward pass (nn.py) ----
def forward(s, X):
    emb = P['E'][X]                                  # (B, T, 16)
    Hf, cf = lstm_fwd(P, 'f_', emb)                  # forward LSTM
    Hb, cb = lstm_fwd(P, 'b_', emb[:, ::-1])         # backward LSTM
    H = np.concatenate([Hf, Hb[:, ::-1]], 2)         # (B, T, 48)
    U = np.tanh(H @ P['Wa'] + P['ba'])               # attention hidden layer
    sc = U @ P['v']; sc -= sc.max(1, keepdims=True)
    alpha = np.exp(sc); alpha /= alpha.sum(1, keepdims=True)
    ctx = (alpha[:, :, None] * H).sum(1)             # weighted summary
    return (ctx @ P['Wo'] + P['bo'])[:, 0]           # logit

def lstm_fwd(P, pre, X):
    Wx, Wh, b = P[pre+'Wx'], P[pre+'Wh'], P[pre+'b']
    B, T, _ = X.shape; h = Wh.shape[0]; XW = X @ Wx + b
    hp = cp = np.zeros((B, h))
    for t in range(T):
        a = XW[:, t] + hp @ Wh
        i = sig(a[:, :h]); f = sig(a[:, h:2*h]); g = np.tanh(a[:, 2*h:3*h]); o = sig(a[:, 3*h:])
        cp = f*cp + i*g; hp = o*np.tanh(cp)          # cell and hidden state
        ...

# ---- detection latency (run.py) ----
def latency(p, K=2, th=0.5):
    for each attack trace: alert = first window j (ending after the onset)
        such that windows j-K+1..j are all predicted attack
        latency = end_position(j) - onset
    for each normal trace: false alarm if any K consecutive windows are flagged
```